

Multideck tenant isolation and release process

A practical operating model for separate customer Supabase projects and safe platform updates.

The decision

Each Multideck customer receives its own fully isolated Supabase project. Their application connects only to that project. Customer projects never connect to one another and no customer sees another customer's data.

What stays separate

Layer	ABC Freight	Genkai	XYZ
Supabase project	Dedicated project	Dedicated project	Dedicated project
Customer data	Only ABC data	Only Genkai data	Only XYZ data
Authentication and storage	Dedicated	Dedicated	Dedicated
Tenant configuration	ABC-specific	Genkai-specific	XYZ-specific

How a Multideck release works

A customer does not update anything themselves. Multideck uses a private release service to apply the same approved core release to each tenant project in a controlled sequence.

1. A feature is completed, tested, and merged into the release branch.
2. The release pipeline prepares the application version and any database migration.
3. A Multideck operator approves the release; it begins with an internal or canary tenant.
4. The release service securely connects to each tenant project, applies the migration, verifies it, and records the result.
5. The matching application version is deployed for that tenant. Any failed tenant is isolated for retry while healthy tenants remain available.

What connects to tenant projects

The customer-facing app uses only that tenant's public Supabase configuration. A separate, private Multideck release service holds the operational deployment access required to run approved migrations. Those credentials are never exposed to the browser or customer users.

Keeping bespoke customer work safe

Every tenant receives the same core Multideck version, but remains free to have its own configuration and approved extensions. A release adds the common feature; it does not reset customer-specific fields or workflows.

Layer	Purpose	Example
Core Multideck	Shared product capabilities	Leads, deals, quotes, security fixes
Tenant configuration	No-code differences per customer	ABC Freight adds a customs-status field
Tenant extension	Approved bespoke capability where needed	ABC-specific report or workflow

Where possible, bespoke needs should be implemented as configurable fields, forms, views, and workflows rather than one-off database forks. This keeps upgrades straightforward while retaining genuine customer flexibility.

Safe migration rules

- Additive first: introduce new fields or tables without immediately removing old ones.
- Compatible releases: new application code works during a short period where tenant databases may be on adjacent versions.
- Canary rollout: test on an internal tenant before the broader rollout.
- Health checks: record the database version, app version, migration outcome, and any failure for every tenant.
- Controlled recovery: pause and retry an affected tenant without rolling back every other customer.

What the internal release dashboard shows

Tenant	Database version	Application version	Status
ABC Freight	Current	Current	Healthy
Genkai	Current	Current	Healthy
XYZ	Previous	Current	Retry required

Recommended starting point

Start with an operator-approved release button rather than deploying every code push automatically. Once the migration runner, health checks, and tenant registry have proven reliable, automate low-risk application releases while keeping schema, security, and destructive changes behind explicit approval.